A BEGINNER'S GUIDE TO GB STUDIO 4.3

Make a Game Boy Shoot 'Em Up

From an empty project to STAR PATROL — a playable vertical shooter with music, sound effects and actor-reuse tricks.



Mock-up of the finished game using the companion asset pack

Written for GB Studio 4.3.x • November 2026

Contents

How to use this guide 4

What you need 4

Conventions 4

1. Installing GB Studio 6

2. A tour of the editor 6

Game World tools 7

3. The five ideas behind every GB Studio game 7

4. The Game Boy's rules (read this once!) 7

5. Art rules and the companion assets 10

The palette 10

What's in StarPatrol_Assets.zip 10

Setting up the sprites in the Sprite Editor 11

6. Sound effects 13

Sound file formats 13

STAR PATROL's sound list 13

7. Music with the built-in Music Editor 14

Anatomy of a song 14

Fast lane: import the ready-made songs 14

Or compose the stage theme yourself 14

Make the title theme 15

Playing music in your game 16

8. The game plan 18

Global variables 18

9. Project settings 18

10. The Title scene 19

11. The stage: a scrolling shooter in five clicks 19

12. Shooting: projectiles and collision groups 20

13. Enemies: the actor pool 20

Step 1 — place four enemies 21

Step 2 — write the logic ONCE as custom scripts 21

Step 3 — hook each enemy up (three tiny scripts per actor) 22

14. Explosions: one actor, every explosion 23

15. Getting hit: lives and invulnerability 24

16. Finish line, Game Over and Mission Complete 24

The finish line 24

Game Over and Mission Complete 25

17. Test, debug and export 25

Test 25

Export your game 26

Final checklist 26

18. The reuse & optimisation cheat sheet 28

19. Troubleshooting 28

20. Where to go next 29

Learn more 29

How to use this guide

This guide takes you from “I've never opened GB Studio” to a finished, playable Game Boy ROM. Every chapter adds one piece to the same project, **STAR PATROL**: a vertical scrolling shooter with a title screen, a long scrolling space stage, a reusable squadron of enemies, explosions, enemy bullets, lives, a score, music on every screen, sound effects for every action, and Game Over / Mission Complete endings.

You don't need to know how to program. GB Studio is a visual tool: you build logic by stacking *events* (“Launch Projectile”, “Play Sound Effect”, “If Variable…”) in lists, and it compiles them into a real ROM that runs on emulators, in a web browser and on actual Game Boy hardware.

What you need

- A Windows, macOS or Linux computer.
- GB Studio 4.3.x (the stable release at the time of writing is 4.3.2, released June 2026). It's free / pay-what-you-want.
- The companion asset pack **StarPatrol_Assets.zip** that came with this PDF: sprites and backgrounds in the correct Game Boy palette, plus both songs as ready-to-import .uge files. You can also draw and compose your own; Chapters 5 and 7 explain how.
- About 2–3 hours for the whole thing. Each chapter ends with something you can play-test.

Conventions

Event scripts are shown in dark boxes that look like GB Studio's event list. The bold part is the event name you add with **+ Add Event**; the text after it is the settings you fill in. Indented lines sit inside an *If* or a *Loop*. Lines starting with # are comments for you, not events.

Example — Scene › On Init

```
# Comment lines like this are just explanations
Play Music Track Song: stage_theme
If Variable Compare With Value $Lives == 0
    Change Scene Scene: Game Over, Fade
```

HEADS UP

GB Studio changes quickly, and event names are occasionally renamed between versions. The fastest way to find any event is to click **+ Add Event** and simply **type part of its name** (e.g. “projectile”, “random”, “shake”) into the search box. If a name in this guide doesn't match exactly, search for the key word.

PART 1

First Steps

Install GB Studio and learn how it thinks

1. Installing GB Studio

1. Go to **gbstudio.dev** (or **chrismaltby.itch.io/gb-studio**) and download the build for your system. Windows has installer and no-installer versions (32- and 64-bit); macOS has Apple Silicon and Intel builds; Linux has AppImage, .deb and .rpm packages, including ARM versions.
2. Install and launch it. You'll see the **New Project** window.
3. Choose the **Empty** template (the Sample Project is great to explore later, but for this guide we start clean). Name it **StarPatrol** and pick a folder you can find easily.
4. Click **Create**. GB Studio opens the **Game World** view with a single scene.

> **TIP**
>
> Your project is a normal folder on disk. Inside it, **assets/** holds your art and audio: **assets/sprites**, **assets/backgrounds**, **assets/music** and **assets/sounds**. Anything you copy into those folders appears in GB Studio automatically — no import button needed.

> **TIP**
>
> Since 4.2, GB Studio has a built-in **Plugin Manager** (menu **Plugins › Plugin Manager**) for installing community plugins, themes and project templates. You won't need any plugins for this guide, but it's where to look once you want extra scene types or events.

2. A tour of the editor

Along the top-left there is a dropdown (the **Project View** button) that switches between the main sections:

Section	What it's for
Game World	Your map of scenes. You place scenes, actors and triggers here and edit their scripts in the right-hand sidebar.
Sprites	The Sprite Editor: animation states, frames and collision boxes for every sprite sheet.
Backgrounds	Preview of background images and their tile counts.
Music	The built-in Music Editor (piano roll + tracker) for .uge songs.
Sounds	Sound effect files you've added (.wav, .vgm, .sav).
Palettes	Colour palettes for Game Boy Color games.
Dialogue Review	All text in your game in one list — handy for spell-checking.
Settings	Project settings: colour mode, music format, default player sprites per scene type, scene-type settings and more.

In the top-right you'll find **Play** (build and run in the built-in emulator) and the **Export** options. The **Debugger** panel shows variables, actors and build output while your game runs — you'll use it in Chapter 17.

Game World tools

The toolbar above the Game World has: **Select** (V), **Add** (actor / trigger / scene), **Eraser**, **Collisions**, **Colors** (GBC only) and **Tile palette**. Hold **Space** and drag to pan; scroll with Ctrl/Cmd to zoom. Hover over a scene and press **P** to set where the player starts.

3. The five ideas behind every GB Studio game

Scenes

A scene is a background image plus everything that lives on it. Each scene has a **Type** that decides how the player is controlled: Top Down 2D, Platformer, Adventure, Shoot 'Em Up, Point and Click, and Logo (a static screen with no player, ideal for title screens because it allows far more unique tiles). You can't change a scene's type while the game is running, so you'll make one scene per “mode”.

Actors

Anything that moves or reacts: enemies, explosions, NPCs, pickups. Actors have a sprite sheet, a position, an optional **Collision Group**, and scripts: **On Init** (runs once when the scene loads), **On Update** (runs over and over), **On Interact**, and **On Hit** (runs when something from a collision group touches it).

Triggers

Invisible rectangles. When the player enters one, its **On Enter** script runs. We'll use one as the finish line.

Scripts, events and custom scripts

A script is a list of events. Scenes, actors, triggers and buttons can all own scripts. A **Custom Script** (the **Scripts** list in the Game World navigator) is a reusable script you **Call** from anywhere — and it can take *parameters* such as “which actor”. That's the heart of our actor-reuse tricks.

Variables

Numbers your game remembers: score, lives, positions. **Global variables** are shared by the whole game. You can rename them (e.g. $Score) by clicking the variable name in any event.

4. The Game Boy's rules (read this once!)

The Game Boy is tiny. GB Studio protects you with limits, and under every scene in the Game World you'll see a bar like **A: 4/20 S: 30/96 T: 1/30** — Actors, Sprite tiles, Triggers. Designing around these limits is why the reuse tricks in this guide matter.

Limit	Value	What it means for a shooter
Actors per scene	20	You can't place 60 enemies in a level. Instead you **recycle** a few (Chapter 13).
Actors visible at once	~10	More than about 10 actors on one screen and some vanish. Keep waves small.
Sprites per scanline	10 hardware sprites	A 16px-wide actor uses two. Five ships side-by-side on one row will flicker. Stagger heights.

Limit	Value	What it means for a shooter
Sprite tiles	96 (fewer if the background is complex)	Reusing one sprite sheet for every enemy costs its tiles only once.
Background unique tiles	128 (more in Logo scenes / Color-only)	A long stage must be built from repeating 8×8 tiles.
Projectiles	5 on screen	Player shots and enemy shots share these slots.
Triggers per scene	30	Plenty for us.
Max scene height	2040 px (255 tiles)	Our stage is 160 × 2040: the longest vertical level allowed.
Audio channels	4 (Duty 1, Duty 2, Wave, Noise)	Sound effects temporarily borrow a music channel.

PART 2

Art & Assets

Sprites, backgrounds and the Game Boy palette

5. Art rules and the companion assets

The palette

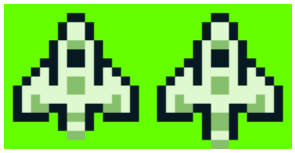
GB Studio reads PNG files and converts them to Game Boy tiles. Your PNGs must use these exact colours:

Colour	Hex	Backgrounds	Sprites
Darkest	#071821	yes	yes
Dark	#306850	yes	**no**
Light	#86c06c	yes	yes
Lightest	#e0f8cf	yes	yes
Transparent (bright green)	#65ff00	no	yes — invisible in game

Sprites are drawn in **16 × 16 frames laid side by side**: a 16×16 image is one frame, 32×16 is two frames, and so on up to 25 frames. Backgrounds must be multiples of 8 pixels in both directions, at least 160 × 144.

What's in StarPatrol_Assets.zip

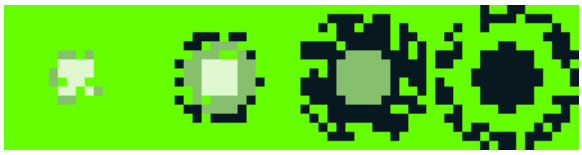
Unzip it and copy the three folders inside **assets/** (sprites, backgrounds, music) into your project's **assets/** folder. The music is covered in Chapter 7. You'll get:



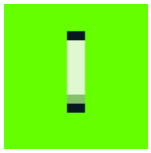
player_ship.png
32×16, 2 frames (engine flicker). Our hero.



enemy_ship.png
32×16, 2 frames. *One* sheet shared by every enemy.



explosion.png
64×16, 4 frames. Used by one recycled explosion actor.



bullet_player.png — the player's laser (projectile).



bullet_enemy.png — the enemies' plasma ball (projectile).

Background	Size	Unique tiles	Used by
title.png	160 × 144	130	Title scene (Logo type allows this many)
stage1.png	160 × 2040	98	The scrolling stage (Shoot 'Em Up)

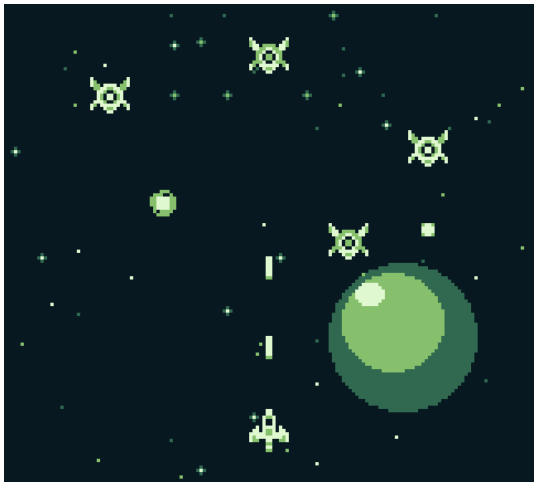
Background	Size	Unique tiles	Used by
gameover.png	160 × 144	53	Game Over scene (Top Down 2D)
victory.png	160 × 144	73	Mission Complete scene (Top Down 2D)

REUSE TRICK

stage1.png is 2040 pixels tall — that's 6,375 tile positions — but only 98 *unique* tiles. The stars are eight little 8×8 tiles scattered over and over, and the space station at the top repeats the same window tile. GB Studio stores each unique tile once, so repetition is free. Use this trick for every long level.



title.png



In-game mock-up (stage + sprites)

Setting up the sprites in the Sprite Editor

Open the **Sprites** section and click each sprite. Two settings matter for a shooter:

- **Collision bounding box** — the invisible box used for hits. Make it a bit *smaller* than the drawing: players forgive near-misses, never unfair hits. Suggested boxes (x, y, width, height in pixels from the frame's top-left): player 4, 4, 8, 9 • enemy 2, 3, 12, 9 • bullet_player 6, 3, 4, 10 • bullet_enemy 5, 5, 6, 6.
- **Animation** — for player_ship, enemy_ship and explosion, make sure all frames are in the default (idle) animation so they loop. Set an animation speed in the actor's properties if it's too fast or slow.

TIP

A sprite sheet type of *Actor* expects separate frames for down/up/right. Our ships are top-down symbols that always look the same, so keep them as simple (non-directional) animated sprites. Fewer frames = fewer sprite tiles.

PART 3

Sound & Music

Sound effects and the built-in Music Editor

6. Sound effects

Every sound in GB Studio is played with the **Play Sound Effect** event. You can pick a built-in preset — **Beep** (with a pitch), **Tone** (with a frequency in Hz) or **Crash** (noise) — or a file you dropped into **assets/sounds**.

Field	What it does
Sound Effect	Preset (Beep / Tone / Crash) or a file from assets/sounds.
Priority	Low / Medium / High. If two effects overlap, the higher priority wins.
Pitch / Frequency	Beep uses a pitch number; Tone uses Hz.
Duration	How long the preset plays (seconds).
Wait until finished	Pauses the script until the sound ends. Leave this OFF for action sounds.
Effect Index	Which effect to play inside an FX Hammer .sav bank.

Sound file formats

- **.wav** — very short, ideally 8-bit mono. Good for a sampled voice (“Go!”).
- **.vgm** — Game Boy VGM files, made in trackers like Furnace. Very flexible.
- **.sav** — an FX Hammer sound bank holding many effects (choose with Effect Index). There are free GB-ready SFX packs on itch.io in .sav and .vgm format.

> **WHY THIS WORKS**
>
> The Game Boy has only four sound channels, and music uses all of them. When a sound effect plays, the sound driver temporarily mutes the music on the channel it needs, then gives it back. That's why a constant machine-gun of noise effects can make your drums disappear — keep shot sounds short and on a different channel than your melody where possible.

STAR PATROL's sound list

We'll use only presets, so you don't need any extra files. Tweak the numbers to taste — the built-in preview button next to the event plays them instantly.

Moment	Preset	Settings	Priority
Player shoots	Beep	Pitch 8, Duration 0.03	Low
Enemy explodes	Crash	Duration 0.25	Medium
Player is hit	Tone	180 Hz, Duration 0.4	High
Press Start on title	Beep	Pitch 6, Duration 0.15	High
Enemy fires	Tone	600 Hz, Duration 0.04	Low

7. Music with the built-in Music Editor

New projects use the **UGE (hUGEDriver)** music format, edited right inside GB Studio. (The legacy alternative, MOD/GBT Player, is set in **Settings › Music Format**; a project can use only one format, so stay on UGE.) GB Studio 4.3 overhauled the editor: MIDI keyboard input, channel solo, exporting songs as MP3/WAV/FLAC, and importing/exporting instruments compatible with hUGETracker.

Anatomy of a song

- **Four channels:** Duty 1 and Duty 2 (square waves — melody and harmony), Wave (custom waveform — great bass), Noise (drums).
- **Patterns:** blocks of 64 rows. Each row can hold a note, an instrument and an effect on each channel. The order list arranges patterns into the full song.
- **Tempo:** the number of ticks per row. Bigger number = slower song.
- **Two views:** *Piano Roll* (click notes with the mouse) and *Tracker* (type notes with the keyboard). Switch with the first icon in the toolbar.

Fast lane: import the ready-made songs

The companion pack includes both songs already composed as **.uge** files — the same format the Music Editor saves:

- **assets/music/stage_theme.uge** — tempo 6, two 64-row patterns: lead melody, off-beat plucks, a bouncing triangle bass and kick/snare/hi-hat.
- **assets/music/title_theme.uge** — tempo 9, a slow melody with an echo on Duty 2 and a long bass line.

1. Copy the **music** folder from the pack into your project's **assets/** folder (next to sprites and backgrounds).
2. Open the **Music** section. Both songs appear in the song list; click one and press Play to hear it.
3. That's it — they now show up in the **Play Music Track** dropdown. Open them in Tracker view to see how they're built, and change anything you like.

> **TIP**
>
> The two **_preview.wav** files that came with the pack are rough renders so you can hear the songs before opening GB Studio. The real Game Boy sound in the Music Editor will be crisper.

> **HEADS UP**
>
> If a .uge file doesn't open in your version, nothing is lost: build the songs by hand with the tables below. They teach you the editor anyway.

Or compose the stage theme yourself

1. Open **Music** and press **+** above the song list. Name the song **stage_theme**.
2. Set **Tempo** to **6** in the right sidebar.
3. Switch to **Tracker** view. Enter the notes in the table below (“A-4” means note A, octave 4). Empty cells stay empty.

4. For instruments: Duty 1 uses instrument 1; Wave uses wave instrument 1; on Noise pick three noise instruments that sound like a kick (low thud), snare (bright burst) and hi-hat (short tick) — click each to preview and adjust its envelope until it sounds right.
5. Press **Play** to hear it loop, then save (Ctrl/Cmd+S).

Row	Duty 1 (lead)	Wave (bass)	Noise (drums)
00	A-4	A-3	kick
04			hat
08	C-5	A-3	snare
12	E-5		hat
16	D-5	F-3	kick
20	C-5		hat
24	B-4	F-3	snare
28	G-4		hat
32	A-4	C-4	kick
36	E-5		hat
40	A-5	C-4	snare
44	G-5		hat
48	F-5	G-3	kick
52	E-5		hat
56	D-5	E-3	snare
60	E-5		hat

TIP

Want it fuller? Copy the Duty 1 notes to **Duty 2** one octave lower (A-3, C-4, …) with a quieter instrument, or add a second pattern and alternate them in the order list. Songs loop automatically. (The imported stage_theme.uge does exactly this.)

Make the title theme

Create a second song called **title_theme**, tempo **9** (slower and moodier), no drums:

Row	Duty 1	Wave		Row	Duty 1	Wave
00	E-5	A-3		32	A-4	C-4
08	D-5			40	C-5	
16	C-5	F-3		44	E-5	
24	B-4			48	A-5	E-3

We'll reuse **title_theme** on the Mission Complete screen too — one less song to store.

TIP

Not a musician? Free GB Studio–ready .uge songs are available on itch.io (search “GB Studio uge”). Drop the .uge file into **assets/music** and it appears in the Music section and in the Play Music Track dropdown. Check the licence first.

Playing music in your game

Add **Play Music Track** to any script and choose a song. **Stop Music** silences it. Put music events in a scene's **On Init** so each screen starts its own track.

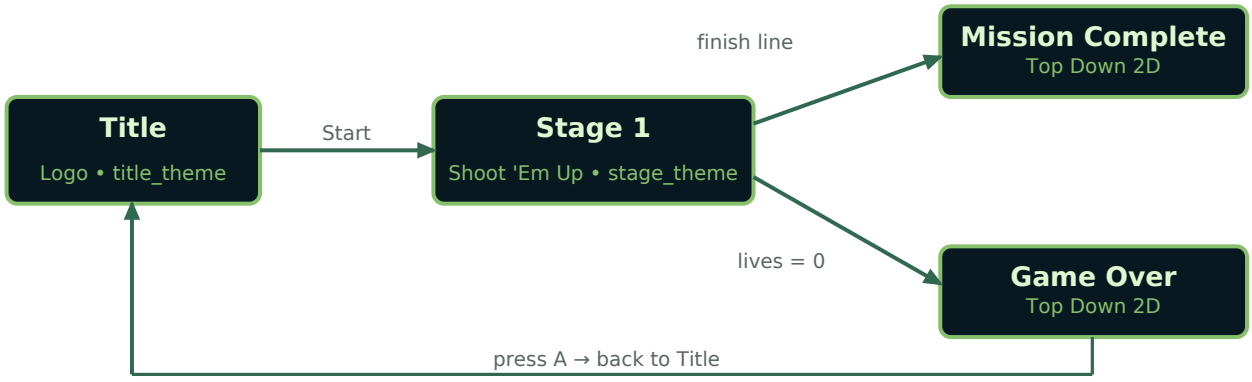
PART 4

Build STAR PATROL

Step by step to a playable vertical shooter

8. The game plan

Here's how the four scenes connect:



Global variables

Open any event that uses a variable and rename these (Variables are listed in the Game World navigator too):

Variable	Purpose
$Score	Points. +10 per enemy.
$Lives	Starts at 3. 0 = Game Over.
$Invuln	1 while the player is flashing after a hit (prevents double hits).
$PX, $PY	Scratch: the player's position.
$EX, $EY	Scratch: an enemy's position.
$RX, $RY	Scratch: a new random spawn position.
$Fire	Scratch: random number deciding if an enemy shoots.

REUSE TRICK

Scratch variables ($PX, $EX, $RX...) are shared by every enemy. They're only used for an instant inside one script, so the four enemies can safely take turns with them. Reusing variables like this keeps your variable list short.

9. Project settings

1. Open **Settings**. Under the colour options, leave the game as monochrome (classic DMG look) for now.
2. Find the **default player sprite** for each scene type and set **Shoot 'Em Up** to **player_ship**.
3. Find the **Shoot 'Em Up** scene-type settings. Set player movement to **lock to the screen edge** (the player slides left/right along the bottom while the stage scrolls — classic Galaga style). Leave other values at their defaults for now; you can tune speed later.
4. Check **Music Format** is **UGE (hUGEDriver)**.

> **TIP**
>
> You can also enable or disable whole scene types in Settings (new in 4.2). Disabling ones you don't use (e.g. Platformer) keeps their engine code out of your ROM.

10. The Title scene

1. In Game World, select the starting scene. Name it **Title**, set **Type: Logo** and **Background: title**.
2. With nothing selected, open the scene's script tab **On Init** and add:

Title › On Init

```
Play Music Track title_theme
Await Input Start, A (search “input”)
Play Sound Effect Beep, Pitch 6, Duration 0.15, High
Fade Screen Out Speed 2
Change Scene Stage 1, Player start at bottom, Direction: Up
```

(You'll be able to choose Stage 1 once you create it in the next chapter — come back and fill this in.) Press **Play**: you should see the title and hear the theme.

11. The stage: a scrolling shooter in five clicks

1. Click **+ Add › Scene** and drop it next to Title. Name it **Stage 1**, **Type: Shoot 'Em Up**, **Background: stage1**. It's a very tall scene — zoom out to see it all.
2. Set the **player start** near the bottom centre: hover at about tile **x 9, y 250** and press **P**.
3. Set the player's start **Direction to Up**. In Shoot 'Em Up scenes the **scroll direction is the direction the player faces when the scene starts** — this is what makes it a vertical shooter instead of a horizontal one.
4. Make sure the Title's Change Scene event points here with Direction: Up as well.
5. Press **Play**. The stars scroll past, the player slides left/right, and the station appears at the top.

Now give the stage its own soundtrack and set up the starting values. Select Stage 1 and open **On Init**:

Stage 1 › On Init

```
# reset the run every time the stage loads
Set Variable To Value $Score = 0
Set Variable To Value $Lives = 3
Set Variable To Value $Invuln = 0
Hide Actor Explosion # add this after Chapter 14
Play Music Track stage_theme
# shooting: see Chapter 12
```

12. Shooting: projectiles and collision groups

Collision groups are how GB Studio decides who can hurt whom. We'll use:

Group	Members	Hits
Group 1	Player's lasers	Enemies (Group 2)
Group 2	Enemy ships	The player (bump damage)
Group 3	Enemy plasma balls	The player

Add to the end of **Stage 1 › On Init**:

Stage 1 › On Init (continued)

```
Attach Script To Button Button: A (Override default: on)
    Launch Projectile Source: Player, Launch At: Direction Up
    # Projectile tab:
    # Sprite: bullet_player Speed: Fastest Life Time: 1.0s
    # Collision Group: Group 1 Collide With: Group 2
    # Destroy On Hit: on
    Play Sound Effect Beep, Pitch 8, Duration 0.03, Low
```

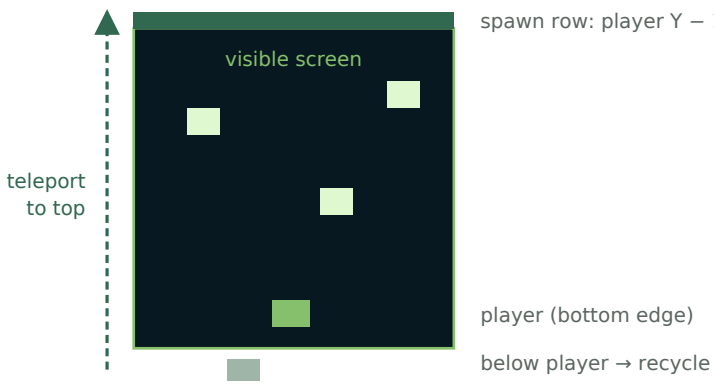
Repeat for button **B** if you like two fire buttons. Play-test: A fires lasers with a pew.

REUSE TRICK

You never create or delete bullets. The engine has a fixed set of projectile slots (5) and recycles them: a projectile that leaves the screen, runs out of Life Time or hits something frees its slot. That's why a short Life Time is good — it frees slots for enemy fire. For advanced control, GB Studio 4.2 added **Load Projectile Into Slot** + **Launch Projectile In Slot**: load the settings once (e.g. in On Init) and launch many times with a lighter event.

13. Enemies: the actor pool

A typical shooter throws dozens of enemies at you. We'll fake that with **just four actors** that recycle themselves: when an enemy is shot or drifts below the screen, it teleports to a random spot just above the top of the screen and flies in again. The player sees an endless stream; the Game Boy sees four actors. This is called **object pooling**, and it's the most important reuse trick in this guide.



The pool: four enemies cycle from below the screen back to just above it.

Step 1 — place four enemies

Add four actors to Stage 1 near the start, all with **Sprite: enemy_ship**, **Collision Group: Group 2**, movement speed 1. Name them Enemy 1–4 and place them at roughly (3, 236), (9, 230), (15, 224), (6, 218) — **staggered vertically** so no two share a row (remember the 10-sprites-per-line limit).

> **REUSE TRICK**
>
> All four enemies use the **same sprite sheet**. GB Studio loads a sprite sheet's tiles once per scene no matter how many actors use it — so four enemies cost exactly the same sprite tiles as one. Check the S: counter under the scene before and after adding enemies 2–4: it doesn't change.

Step 2 — write the logic ONCE as custom scripts

Instead of copying the same events into four actors (and fixing bugs four times), create **Custom Scripts** (Game World navigator › Scripts › +). Give each one an **actor parameter** called **Enemy**: when you add events inside, choose **Enemy** in the actor dropdown instead of a specific actor.

Custom Script: Enemy Respawn (parameter: Actor “Enemy”)

```
Store Actor Position In Variables Player → $PX, $PY
Math Functions $RX = Random, min 1, max 18
Evaluate Math Expression $RY = $PY - 18
If Variable Compare With Value $RY < 8
    # we're near the end of the level: retire this enemy
    Deactivate Actor Enemy
Else
    Set Actor Position Enemy, X: $RX, Y: $RY (units: tiles)
    Show Actor Enemy
```

To use a variable as a position, click the small type toggle next to the X/Y field and switch it from a number to a variable.

Custom Script: Enemy Think (parameter: Actor "Enemy")

```
Actor Move Relative Enemy, X: 0, Y: +2 tiles, Lock Direction: on
# random 1-in-6 chance to shoot
Math Functions $Fire = Random, min 0, max 5
If Variable Compare With Value $Fire == 0
    Launch Projectile Source: Enemy, Direction: Down
    # Sprite: bullet_enemy Speed: Slow Life Time: 2s
    # Collision Group: Group 3 Collide With: Player
    Play Sound Effect Tone, 600 Hz, 0.04, Low
# has it slipped below the player? recycle it
Store Actor Position In Variables Enemy → $EX, $EY
Store Actor Position In Variables Player → $PX, $PY
If Variable Compare With Variable $EY > $PY
    Call Script Enemy Respawn (Enemy = Enemy)
```

Custom Script: Enemy Destroyed (parameter: Actor "Enemy")

```
Play Sound Effect Crash, Duration 0.25, Medium
Variables: Add $Score + 10
Hide Actor Enemy
Call Script Show Explosion (Enemy = Enemy) # Chapter 14
Call Script Enemy Respawn (Enemy = Enemy)
```

Step 3 — hook each enemy up (three tiny scripts per actor)

Select **Enemy 1** and fill its tabs. Then do exactly the same for Enemies 2–4 — each just calls the shared scripts with **Self**:

Enemy N › On Update

```
Call Script Enemy Think (Enemy = Self)
```

Enemy N › On Hit › Group 1 (hit by the player's laser)

```
Call Script Enemy Destroyed (Enemy = Self)
```

Enemy N › On Hit › Player (rammed the player)

```
Hide Actor Self
Call Script Enemy Respawn (Enemy = Self)
# the damage itself is handled by the scene (Chapter 15)
```

REUSE TRICK

Once Enemy 1 is fully set up, you can **copy and paste the actor** (right-click › Copy, then Paste in the scene) to make Enemies 2–4 with all scripts included. Then just move them. Because the scripts only say **Self**, every copy works without edits.

Play-test. Enemies drift down, fire occasionally, explode in a burst of noise when shot (the explosion graphic comes next) and reappear from the top. Shoot ten and you've seen 10 “different” enemies from four actors.

WHY THIS WORKS

Actors that are far off-screen are paused by the engine to save CPU. That's exactly why we respawn enemies *just* above the visible area: as the camera scrolls up, they come into view and wake up. If you spawn them too far above (say 40 tiles), they'll simply arrive later.

14. Explosions: one actor, every explosion

Each explosion could be its own actor — but we only have 20 actor slots. Instead, one **Explosion** actor jumps to wherever something blows up, plays its animation, and hides again.

1. Add an actor named **Explosion** with **Sprite: explosion**, no collision group. Place it anywhere near the player's start.
2. Make sure Stage 1 › On Init contains **Hide Actor: Explosion** (Chapter 11).
3. Create a custom script:

Custom Script: Show Explosion (parameter: Actor “Enemy”)

```
Store Actor Position In Variables Enemy → $EX, $EY
Set Actor Position Explosion, X: $EX, Y: $EY
Activate Actor Explosion
Set Actor Animation State Explosion, Default, Loop: off
Show Actor Explosion
Wait 0.25 s
Hide Actor Explosion
```

REUSE TRICK

Because Show Explosion contains a Wait, the enemy's script pauses for a quarter-second before the enemy respawns. That small pause is actually nice: the enemy doesn't pop back in the same instant it died. If two enemies explode within 0.25 s of each other, the single explosion simply jumps to the newest one — on a busy Game Boy screen nobody notices. Want overlapping explosions? Add an Explosion 2 and alternate between them with a variable.

15. Getting hit: lives and invulnerability

Player damage lives on the **scene**, not on individual enemies. Select Stage 1 and find its **On Player Hit** tabs (one per collision group). Group 2 (enemy bodies) and Group 3 (enemy bullets) both call the same custom script:

Custom Script: Player Damaged

```
If Variable Compare With Value $Invuln == 1
    # still flashing — ignore this hit
    Stop Script
Set Variable To Value $Invuln = 1
Play Sound Effect Tone, 180 Hz, 0.4, High
Camera Shake 0.3 s
Decrement Variable $Lives
If Variable Compare With Value $Lives == 0
    Stop Music
    Wait 0.5 s
    Fade Screen Out
    Change Scene Game Over
Else
    Actor Effects Flicker, Actor: Player, Duration: 1.5 s
    Set Variable To Value $Invuln = 0
```

Stage 1 › On Player Hit › Group 2 and › Group 3

```
Call Script Player Damaged
```

If your version doesn't have **Stop Script**, put everything after the first If into its **Else** branch instead. Play-test: get hit, hear the low buzz, see the shake and the flicker, and after three hits you're sent to Game Over (build that next).

> **WHY THIS WORKS**
>
> The Game Boy plays sound effects by borrowing a channel from the music. Giving the hit sound **High** priority guarantees it is heard over the constant laser beeps (Low).

16. Finish line, Game Over and Mission Complete

The finish line

When the camera reaches the top of the scene it stops scrolling, and the player ends up in the lower part of the station screen. Add a **Trigger** across the full width (20 tiles) from about **y 12 to y 17**:

Finish Trigger › On Enter

```
Stop Music
Play Sound Effect Beep, Pitch 7, Duration 0.4, High
Fade Screen Out
Change Scene Mission Complete
```

The respawn script already retires enemies near the top (the $RY < 8 check), so the last screen is calm.

Game Over and Mission Complete

Add two more scenes, both **Type: Top Down 2D** (Logo scenes can't show dialogue), with backgrounds **gameover** and **victory**. They need a player start position, but we'll hide the player.

Game Over › On Init

```
Hide Actor Player
Display Dialogue “FINAL SCORE: $Score” ↵ “Press A to retry”
Fade Screen Out
Change Scene Title
```

Mission Complete › On Init

```
Hide Actor Player
Play Music Track title_theme # reused song!
Display Dialogue “WELL DONE, PILOT!” ↵ “SCORE: $Score”
Fade Screen Out
Change Scene Title
```

To put a variable in dialogue, type **$** in the text box and pick **Score** from the pop-up list. The dialogue waits for A automatically. Since Title → Stage 1 resets $Score and $Lives in the stage's On Init, every new run starts fresh.

17. Test, debug and export

Test

- **Play** builds and runs the game in GB Studio's emulator. Keys: arrow keys / WASD to move, Z or J = A, X or K = B, Enter = Start.
- Open the **Debugger** while playing to watch $Score, $Lives and actor positions change live. If an enemy never comes back, watch $RY: if it's below 8, the enemy was retired.
- Use the scene's **A/S/T** counter to stay within limits: you should have 6 actors (4 enemies, Explosion, plus the player).

Export your game

Use **Game › Export As…** (or the export menu in the toolbar):

Export	Gives you	Play it on
ROM	a .gb (or .gbc) file	Any Game Boy emulator (SameBoy, mGBA, Emulicious…) or a flash cart on real hardware
Web	an HTML5 folder	Upload to itch.io as an HTML game — playable in a browser, even on phones
Pocket	an Analogue Pocket build	Analogue Pocket

> **TIP**
>
> Upload the Web export as a zip to itch.io, tick “This file will be played in the browser”, and set the viewport to something like 480 × 432 (3× scale). Congratulations — STAR PATROL is published!

Final checklist

✓	Item
☐	Title shows, plays title_theme, Start/A goes to Stage 1 (Direction Up)
☐	Stage 1 scrolls upward, stage_theme plays
☐	A fires lasers with a beep (Group 1 → Group 2)
☐	4 enemies, same sprite, Group 2, each calls Enemy Think / Destroyed / Respawn with Self
☐	One Explosion actor hidden on start, reused by Show Explosion
☐	Enemy bullets in Group 3 hit the player
☐	On Player Hit (Groups 2 & 3) → Player Damaged; 3 lives; flicker
☐	Finish trigger → Mission Complete; lives 0 → Game Over; both show score and return to Title

PART 5

Tricks & Next Steps

Reuse everything, fix problems, grow your game

18. The reuse & optimisation cheat sheet

Trick	Why it helps	Where we used it
Actor pooling	Unlimited enemies from a fixed number of actors (limit 20 per scene, ~10 on screen).	Enemies 1–4 respawn above the screen
Shared sprite sheets	A sheet's tiles load once per scene, however many actors use it.	All enemies use enemy_ship
One effect actor, many effects	Saves actor slots and sprite tiles.	The Explosion actor
Custom scripts with actor parameters	Logic written once; fix a bug in one place. Copies of an actor just pass Self.	Enemy Think / Destroyed / Respawn
Copy & paste actors	Duplicates scripts, sprite and settings in one go.	Enemies 2–4
Scratch variables	A small set of temp variables used by many scripts keeps the list tidy.	$PX, $EX, $RX, $Fire
Repeating background tiles	Unique tiles are what count. A 2040px-tall level can use under 100 tiles.	stage1.png
Projectile slots	Short Life Times free slots; Load/Launch In Slot (4.2+) avoids re-sending settings.	Player & enemy shots
Reuse songs	Music takes ROM space; tempo and context make the same tune feel new.	title_theme on the victory screen
Preset sound effects	Beep/Tone/Crash need no files and no ROM space for samples.	All SFX
Deactivate vs Hide	Hide keeps an actor running but invisible; Deactivate stops its scripts and frees the CPU.	Retired enemies near the finish
Stagger actors vertically	Avoids the 10-sprites-per-scanline flicker.	Enemy start positions
Smaller collision boxes	Feels fairer, and tiny boxes on bullets look more precise.	Sprite Editor settings

19. Troubleshooting

Problem	Likely cause & fix
The stage scrolls sideways	The player's start direction isn't Up. Set it on the scene's player start and in Title's Change Scene event.
Lasers pass through enemies	Projectile Collide With must include Group 2, and the enemies' Collision Group must be Group 2. Check the bounding boxes aren't zero-sized.
Enemies never come back	$RY went below 8 (that's the end-of-level retirement), or the respawn row is far above the screen. Watch it in the Debugger.
Enemies vanish / flicker	Too many actors on one screen or on one horizontal line. Stagger them; keep waves to 4–5.
Player loses several lives at once	$Invuln isn't set to 1 before the flicker, or the hit script runs from both enemy and scene in a loop. Use one Player Damaged script.
Music drops out during shooting	Shot sounds borrow a channel. Use shorter durations, Low priority, or a different preset.

Problem	Likely cause & fix
Sprites look wrong / have a green box	Use #65ff00 for transparency and avoid #306850 in sprites.
“Too many tiles” warning	Your background has more than 128 unique tiles in a non-Logo scene. Reuse tiles, or use Logo type for still screens.
An event in this guide doesn't exist	Search the Add Event menu for a key word. Names change between versions.

20. Where to go next

- **A boss.** Use a scene with a large background-drawn boss and a few actors as its weak points; reuse the Explosion actor for its death sequence.
- **Power-ups.** Give Enemy Destroyed a 1-in-10 chance to place a pickup actor. On Hit › Player: set $Power = 1, and in the A-button script launch two projectiles at angles when $Power is 1.
- **Free movement.** Switch the Shoot 'Em Up setting to free movement. Then calculate the spawn row from the camera instead of the player (since 4.2/4.3, script values can read camera properties, including scroll position).
- **Waves.** Use the 4.3 **Data Table Lookup** event to read enemy patterns (x position, speed) from a table you can import/export as CSV.
- **Colour.** Switch the colour mode to Color + Monochrome and paint palettes with the Colors tool — the game still runs on the original Game Boy.
- **Better audio.** Grab a free FX Hammer .sav bank for punchier explosions, and try the new MIDI input in the Music Editor.

Learn more

- Official documentation: **gbstudio.dev/docs**
- Changelog for each release: **github.com/chrismaltby/gb-studio** (CHANGELOG.md)
- Articles and tutorials: **gbstudiocentral.com**
- The community Discord (linked from gbstudio.dev) — friendly and full of shmup makers.

About this guide: written for GB Studio 4.3.x in November 2026 using the official documentation and changelog. STAR PATROL's art and music in the companion pack were made for this guide and is free to use in your own games.